

Verifica automatica di programmi

Riccardo Torre

9 aprile 2026

Indice

1	Alcune note iniziali	1
2	Una panoramica sulla verifica di programmi	1
2.1	Annotazioni	6
2.1.1	Esempio di LinearSearch	10

Lezione del giorno: 02/03/2026

1 Alcune note iniziali

Il primo esame è composto da 2 prove parziali, ciascuna vale 25% prova iniziale + 25% prova finale. Il restante 50% è riservato ad un progetto individuale

$$25\%PI + 25\%PF + 50\%P$$

Gli altri appelli (2,3,4 e 5) sono 100% esame scritto (no progetto).

Il corso finisce circa a metà maggio. Il primo appello è all'inizio di giugno. Il progetto viene assegnato circa a metà del semestre e comprende una relazione. Il libro di testo è di *Aanon Brandley & Zohan Manna* che usa un linguaggio di programmazione ideale che si chiama π che è simile al C ma semplificato. **Rust** sarà presente in questo corso. Verus (della Microsoft) e Why (Rust Belt). La verifica Rust (collegli di Parigi che insegnano codifica Rust).

2 Una panoramica sulla verifica di programmi

Floyd e Hoare furono i pionieri di questa teoria alla fine degli anni 60. Il limite fondamentale per la verifica automatica di programmi resta sempre lo stesso: non possiamo creare un verificatore universale che decida la correttezza semantica per programmi arbitrari[itf]. Questa condizione viene garantita dal teorema 1.

Teorema 1 (di Rice). *Il teorema di Rice stabilisce che tutte le proprietà non banali delle funzioni calcolabili da programmi (tranne quelle sempre vere o sempre false) sono indecidibili, ovvero non esiste un algoritmo generale che, dato un programma, possa determinare automaticamente se soddisfa tale proprietà.[Wik]*

In pratica, si aggirano questi limiti con approssimazioni, analisi statiche conservative, o restrizioni sul dominio (es. programmi finiti o lineari), ma la verifica totale resta impossibile [itf]. Esistono degli strumenti per effettuare quella che viene chiamata *analisi statica* per fare la dimostrazione di correttezza di un programma a partire dal suo codice sorgente in modo quasi totalmente meccanico. Ci sono tre tecniche fondamentali che andremo a studiare in questa prima parte del corso:

1. **La specifica:** specificazione di un programma. Quello che un programma deve fare (caricare qualcosa su un sito, ordinare un array) viene descritto con formule in **logica del 1° ordine**. Ad esempio per dire che un array è ordinato tra l'indice l e l'indice u bisogna dire

$$\text{sorted}(a, l, u) \equiv \forall i \forall j: l \leq i \leq j \leq u \implies a[i] \leq a[j]$$

La logica composizionale da sola non è sufficientemente espressiva. Avremo bisogno di scrivere per ogni funzione¹ di un programma bisogna specificare:

- (a) **Pre-condizione:** è un qualcosa che si assume vero prima che inizi la computazione della funzione. Contiene anche ciò che la funzione assume per poter lavorare. Descrive lo stato del programma prima dell'esecuzione della funzione. Utilizzeremo la parola chiave “*requires*” cioè ciò che la funzione richiede prima della sua esecuzione.
- (b) **Post-condizione:** ciò che vale dopo l'esecuzione della funzione. Descrive lo stato del programma dopo la computazione della funzione. Utilizzeremo la parola chiave “*ensures*” cioè ciò che la funzione assicura dopo la sua esecuzione.

L'esecuzione di una funzione modifica lo stato di un programma.

Le annotazioni (la categoria generale a cui appartengono le pre-condizioni e le post-condizioni) sono formule incorporate nel codice e si distinguono nel modo seguente:

- **pre-condizioni:** formule che devono valere prima dell'esecuzione della funzione;
- **post-condizioni:** formule che devono valere dopo l'esecuzione della funzione;
- **asserzioni:** formule contenute nel corpo della funzione che devono valere in quel punto nel codice.

Le annotazioni sono riconoscibili dentro il codice perché riportano la chiocciola @L²: F³
Quindi da un programma

```
<type> foo(<type_1> param_1 ... <type_n> param_n)
{
    <block_1>
    <block_2>
    return( )
}
```

si ottiene un programma annotato

```
@pre: F
@post: G
<type> foo(<type_1> param_1 ... <type_n> param_n)
```

¹ogni programma può essere visto come una collezione di funzioni.

²L è una label.

³formula di logica del primo ordine che deve essere vera nello stato del programma.

```

{
    <block_1>
    @L: H
    <block_2>
    return(  )
}

```

C'è una branca dell'informatica che studia come generare automaticamente le annotazioni. In questo corso non riusciamo a vedere questa parte e le scriveremo sempre manualmente. La formula H deve essere vera nello **stato del programma** a quel punto (linea interessata da $@L: H$).

Stato del programma È definito dall'indirizzo di programma nel PC⁴ e le variabili del programma (ad esempio $x : int, y : int, i : int, j : int, a : int[]$). Lo stato è una fotografia della memoria nell'esecuzione.

```

PC = 0010011
x = 5
y = 0
j = -2
a = [0,0,3,0,17,-1]

```

Quindi lo stato del programma può essere visto come un assegnamento di valori del tipo richiesto al PC e a ogni variabile (parametri) del programma.

```

@pre: F
@post: G
<type> foo(<type_1> param_1, <type_n> param_n)
{
    <block_1>
    @L: H
    <block_2>
    return(  )
}

```

Se la funzione `foo` viene chiamata con `foo(a,0,16)` vuol dire che $param_1 = a$, $param_2 = 0$, $param_3 = 16$. I **parametri** (o “*parametri formali*”) sono quelli che definiscono la funzione, gli **argomenti** (o “*parametri attuali*”) sono quelli della chiamata di funzione. Un parametro può essere passato **per valore** (la funzione non può modificarlo; tutti i parametri sono passati per valore nel linguaggio `pi`) o **per riferimento** (il valore può essere modificato dalla funzione).

2. **Metodo delle asserzioni induttive:** è un metodo per dimostrare **proprietà di correttezza parziale** (o in generale “*safety*”) ovvero che il programma non va in stato d'errore. Se un'asserzione è vera nello stato 0, e nello stato n , allora deve essere anche nello stato $n + 1$. Questa è solo l'intuizione dell'induzione ben fondata utilizzata nel metodo delle asserzioni induttive.

- **Funzione parzialmente corretta:** una funzione è *parzialmente corretta* se per ogni ingresso che soddisfa la pre-condizione, se la funzione termina, allora l'uscita soddisfa

⁴program counter.

la post-condizione⁵. Si dice “*parzialmente*” perché si ammette la terminazione della funzione. Il termine “*corretta*” viene usato perché pre e post-condizione sono valide.

- **Funzione totalmente corretta:** una funzione è *totalmente corretta* se per ogni ingresso che soddisfa la pre-condizione, la funzione termina e l'uscita soddisfa la post-condizione.

3. **Metodo delle funzioni d'ordine:** viene usata sui valori e sulla ricorsione. Viene usata per la totale correttezza, quindi per dimostrare la terminazione. **Funzione d'ordine** perché tipicamente la terminazione viene dimostrata dando una funzione definita come un'espressione costruita sulle variabili di un programma che mappa l'espressione ad un valore, in un dominio, con un ordinamento ben fondato (per esempio i numeri naturali). Si dimostra che, ad ogni chiamata ricorsiva o ad ogni iterazione/chiamata ricorsiva del ciclo, l'espressione diventa più piccola. In un ordinamento ben fondato, per esempio su i numeri naturali, se si definisce una funzione su un dominio con ordinamento ben fondato in termini di variabili di programma e si mostra che ad ogni iterazione l'espressione diventa più piccola, siccome lo zero è il fondo dell'ordinamento ben fondato, non potrà scendere all'infinito e quindi il ciclo terminerà.

Le specifiche non sono completamente automatizzate mentre i metodi delle asserzioni induttive e delle funzioni d'ordine sì.

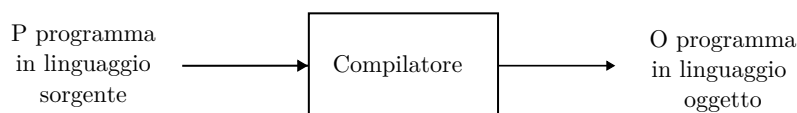


Figura 1: compilatore.

In un linguaggio di programmazione, un compilatore (fig. 1) prende in ingresso un programma P in un linguaggio sorgente e restituisce un programma O nel linguaggio oggetto.

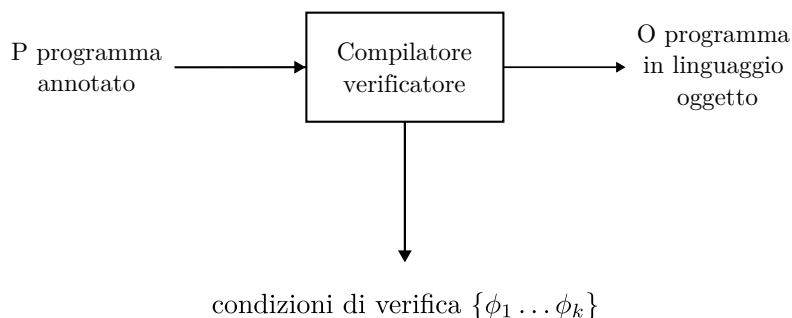


Figura 2: compilatore verificatore.

Un compilatore verificatore (fig. 2) prende in ingresso un programma P annotato (pre-condizioni, post-condizioni, asserzioni e programma nel linguaggio sorgente di prima) e restituisce in uscita:

1. un programma O nel linguaggio oggetto del compilatore precedente;

⁵Nota: stiamo parlando degli ingressi e uscite delle funzioni di un programma, non dell'ingresso e dell'uscita di un programma.

2. delle formule $\{\phi_1 \dots \phi_k\}$ dette **condizioni di verifica** che sono formule la cui validità assicura che il programma è *parzialmente corretto* se si usa solo il *metodo delle asserzioni induttive* e *totalmente corretto* se si usa anche il *metodo delle funzioni d'ordine*. Il metodo delle asserzioni induttive e il metodo delle funzioni d'ordine sono *pienamente automatizzabili*, mentre la specifica no.

Questi compilatori implementano i metodi delle asserzioni induttive e delle funzioni d'ordine. Purtroppo non sono completamente automatizzabili come in fig. 3 per il teorema 1.

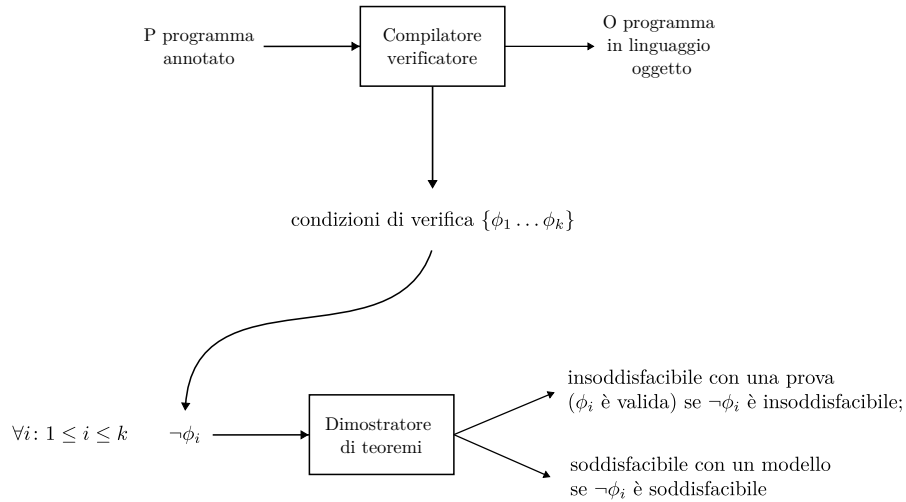


Figura 3: dimostratore di teoremi nel caso ideale.

$\forall i: 1 \leq i \leq k$ si ha che $\neg\phi_i$ in un dimostratore di teoremi che sia una **procedura di decisione** risponde con:

- insoddisfacibile con una prova (ϕ_i è valida) se $\neg\phi_i$ è insoddisfacibile;
- soddisfacibile con un modello se $\neg\phi_i$ è soddisfacibile.

Nella logica del 1° ordine, l'insoddisfacibilità o equivalentemente la validità è solo semi-decidibile.

Un dimostratore di teoremi che prende in ingresso $\neg\phi_i$ restituisce:

- insoddisfacibile⁶ con prova se $\exists i. \neg\phi_i$ è insoddisfacibile ($\forall i. \phi_i$ è valida⁷);
- ? se $\neg\phi_i$ è soddisfacibile.

Avendo a disposizione una **procedura di decisione**, vorremmo che il dimostratore rispondesse:

- insoddisfacibile con prova se $\exists i. \neg\phi_i$ è insoddisfacibile;
- soddisfacibile con un modello se $\neg\phi_i$ è soddisfacibile.

⁶significa che è sempre falsa.

⁷è sempre vera.

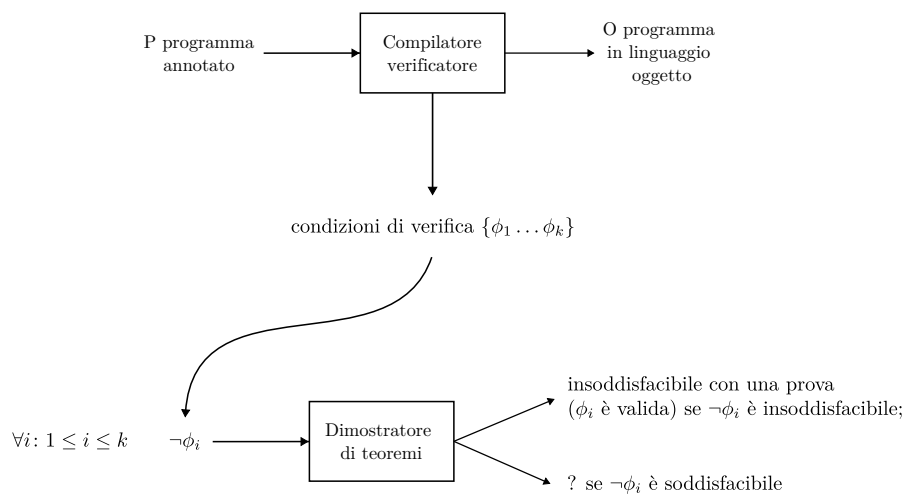


Figura 4: il dimostratore di teoremi nella logica del 1° è semidecidibile.

Se $\neg\phi_i$ è soddisfacibile, vuol dire che ϕ_i , che è una formula restituita dal compilatore-verificatore, è in-valida (cioè non valida). Avere un modello permette di ricostruire, in fase di debugging, lo stato di non validità del programma quando ϕ_i è in-valida.

Lezione del giorno: 05/03/2026

2.1 Annotazioni

Le annotazioni sono delle formule logiche scritte dentro il codice stesso. Possono essere di 4 tipi:

1. **Pre-condizioni di funzione:** ciò che la funzione richiede.
2. **Post-condizioni di funzione:** ciò che la funzione assicura.
3. **Invarianti di ciclo:** rappresenta quello che è vero durante tutta l'esecuzione del ciclo. Invariante nel senso che non varia il valore di verità della formula logica, che è vera prima, durante e dopo il ciclo.
4. **Asserzioni:** formule generali. Vi sono:
 - (a) **Asserzioni per analizzare le funzioni ricorsive:** non posso usare le precedenti, sono necessarie questa tipologia di asserzioni.
 - (b) **Asserzioni al Runtime:** non sempre si possono fare controlli a livello statico. Sono controlli a livello di esecuzione (a livello dinamico).

Restrizioni sintattiche Le formule sono in logica del I ordine. In una formula della logica del I ordine vi sono variabili, costanti, quantificatori esistenziali e universali, funzioni, predicati e relazioni (come $>$, $<$, $\leq$, $\geq$, $=$, $\dots$). In ciascuna formula possono essere distinte variabili libere e variabili quantificate.

$$\begin{array}{c}
\text{variabili quantificate} \\
\swarrow \quad \downarrow \\
sorted(a, l, u) \iff \forall i, \forall j: l \leq i \leq j \leq u \implies a[i] \leq a[j] \\
\searrow \quad \swarrow \\
\text{variabili libere}
\end{array}$$

Figura 5: esempio di formula in logica del I ordine.

Esempio di formula in logica del I ordine Un array è ordinato tra l'indice l e l'indice u se rispetta la formula in fig. 5. Nelle formule le variabili in senso logico (cioè tutto quello che non è una costante, né una relazione, né una funzione, né un connettivo) possono essere quantificate o libere. Le annotazioni sono formule di logica del I ordine dove le sole variabili libere sono le variabili del programma (inclusi i parametri delle funzioni) ⁸. Non vogliamo che entri una variabile che non ha a che fare con il programma e non sappiamo che valore possa assumere.

Lo stato del programma include un assegnamento di valori dei tipi appropriati alle variabili. Solo le variabili libere sono variabili del programma (quindi non sono comprese in questo caso le variabili quantificate) e a cui vengono assegnati valori che riflettono lo stato del programma.

- **Pre-condizione di una funzione:** è una formula della logica del I ordine dove le sole variabili libere sono i parametri della funzione.
- **Post-condizione di una funzione:** è una formula della logica del I ordine dove le sole variabili libere sono i parametri della funzione e la variabile rv che rappresenta il valore restituito.
- **Invariante di ciclo e altre asserzioni:** è una formula della logica del I ordine dove le sole variabili libere sono le variabili ⁹ della funzione dove il ciclo compare e i parametri della funzione (se servono).

```

@pre: T
diventa @pre: 0 ≤ l ∧ u < |a|
@post: T
diventa @post: rv = true ⇔ ∃i: l ≤ i ∧ i ≤ u ∧ i ≤ u ∧ a[i] = e
bool LinearSearch(int[] a, int l, int u, int e)
{
  @I: l ≤ i ∧ ∀k: (l ≤ k < i ⇒ a[k] ≠ e)
  for(int i := l; i ≤ u; i := i+1){
    @E: 0 ≤ i < |a|
    if(a[i]=e)
      return true;
  }
  return false;
}

```

⁸variabili globali (se il linguaggio lo ammette), variabili locali e parametri delle funzioni

⁹si intende variabili locali perché nel linguaggio PI e per semplicità non verranno trattate le variabili globali.

Nota Se mettessimo l'if: `if(1 < 0  $\vee$  u  $\geq$  |a|) return false;` allora non avremmo più bisogno di `@pre`. Solitamente le `@pre`: T banali sono date solo per le funzioni pubbliche, ma ai fini di significatività dell'esempio non si mette l'if. In tal caso, vale che `@post` è garantita quando `@pre` è valida, altrimenti non si dice nulla sul suo comportamento.

Invarianti di ciclo nel while e nel for Per un ciclo del tipo

```
while(<cond>){
    <body>
}
```

l'invariante `@I:F` garantisce che la formula `F` è vera all'entrata: quando si entra nel corpo, `F  $\wedge$  cond` è vero, mentre quando si esce dal ciclo, `F  $\wedge$   $\neg$  cond` è vero. Al termine di ogni iterazione, è sempre vera `F`. Per un ciclo del tipo

```
for(<init>; <cond>; <inc>){
    <body>
}
```

con l'invariante `@I:F`, lo si trasforma in

```
<init>
@I:F
while(<cond>){
    <body>
    <inc>
}
```

e si ragiona in modo analogo a while, come prima.

Asserzioni a tempo di esecuzione Le asserzioni a tempo d'esecuzione (`@E`) servono a proteggere da 3 tipi di errori al runtime:

- **Divisione per zero e resto per la divisione per zero:**

```
@E: y  $\neq$  0
z := x div y;            $\leftarrow$  divisione intera
z := x mod y;           $\leftarrow$  resto della divisione intera
```

se staticamente possiamo dimostrare che `y  $\neq$  0`, allora siamo sicuri ed `@E` non è necessaria. Se tale dimostrazione non riusciamo a darla, il controllo `y  $\neq$  0` deve essere eseguito a tempo d'esecuzione ed è richiesto al compilatore dall'annotazione `@E` che introduce codice che esegue il controllo e gestisce l'errore.

- **Array out of bounds:**

```
@E: n  $\geq$  0  $\wedge$  n < |a|
a[m] := x;
```

Nota in `LinearSearch`, l'asserzione `@E` non era necessaria perché il programma è già protetto da `@pre`, ma non succede niente.

- **Dereferenzamento da null:**

```
@E: p  $\neq$  null
*p := x;
```


Nota Tutte queste asserzioni sono generabili automaticamente dal compilatore, a differenza delle invarianti di ciclo.

Lezione del giorno: 16/03/2026

Stiamo dimostrando la correttezza parziale, per ogni funzione.

Correttezza parziale Per ogni ingresso (valori dati ai parametri¹⁰ durante la chiamata di funzione) che soddisfi la pre-condizione, **se la funzione termina**, allora la post-condizione è soddisfatta.

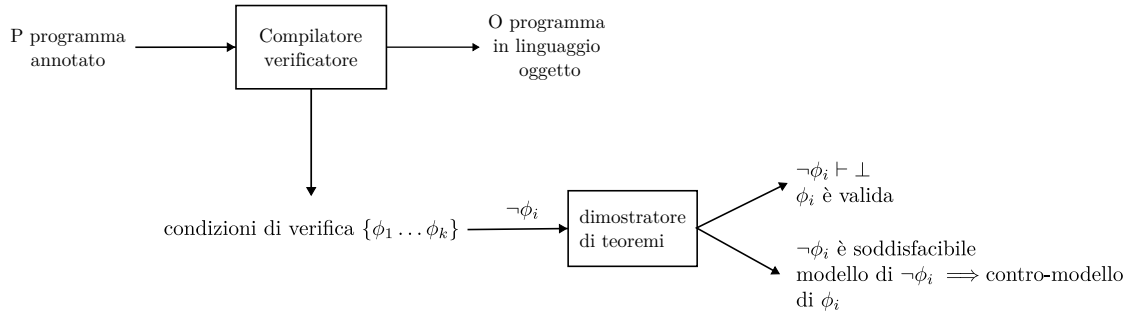


Figura 6: un compilatore-verificatore che genera formule per il dimostratore di teoremi. Il dimostratore di teoremi risponde con una prova che ϕ_i è soddisfatta se da $\neg\phi_i$ si arriva ad una contraddizione, e risponde con un modello quando $\neg\phi_i$ è soddisfacibile da cui è possibile costruire un contro-modello di ϕ_i . Questo serve per catturare gli stati del programma in cui ϕ_i è violata e dato che lo stato di un programma è l'assegnamento dei valori alle variabili del programma, è possibile ricavare informazioni utili per andare a correggere la logica del programma che causa la violazione di ϕ_i .

1. **Annotazioni:** includono le pre-condizioni, post-condizioni, invarianti di ciclo e asserzioni al runtime.
2. **Cammini di base:** i programmi contengono iterazioni e ricorsioni. Questa tecnica permette di decomporre il testo di un programma in cammini e fare l'analisi su ciascuno di essi. Un **cammino di base**¹¹ inizia con una precondizione o un'invariante di ciclo e termina con una post-condizione o un'invariante di ciclo o un'asserzione di altro tipo. In mezzo ci sono istruzioni.

Cammino di base

```

@F  → pre-condizione o invariante di ciclo
{
  S1;
  S2;
  ...
  Sn;
}  istruzioni assume o assegnamenti alle variabili
@G  → post-condizione, invariante di ciclo o asserzione di altro
    tipo
  
```

Le $S_1; \dots; S_n$ sono istruzioni *assume* o assegnamenti alle variabili della funzione.

¹⁰ovvero gli argomenti.

¹¹questa definizione va bene sia per l'iterazione che per la ricorsione.

2.1.1 Esempio di LinearSearch

```

@pre:  $0 \leq l \wedge u < |a|$ 
@post:  $rv = \text{true} \iff \exists i: l \leq i \leq u \wedge a[i] = e$ 
Bool LinearSearch(int[] a, int l, int u, int e)
{
    @L:  $l \leq i \wedge (\forall j: l \leq j < i \implies a[j] \neq e)$ 
    for(int i := l; i ≤ u; i := i+1){
        if(a[i]=e)
            return true;
    }
    return false;
}

```

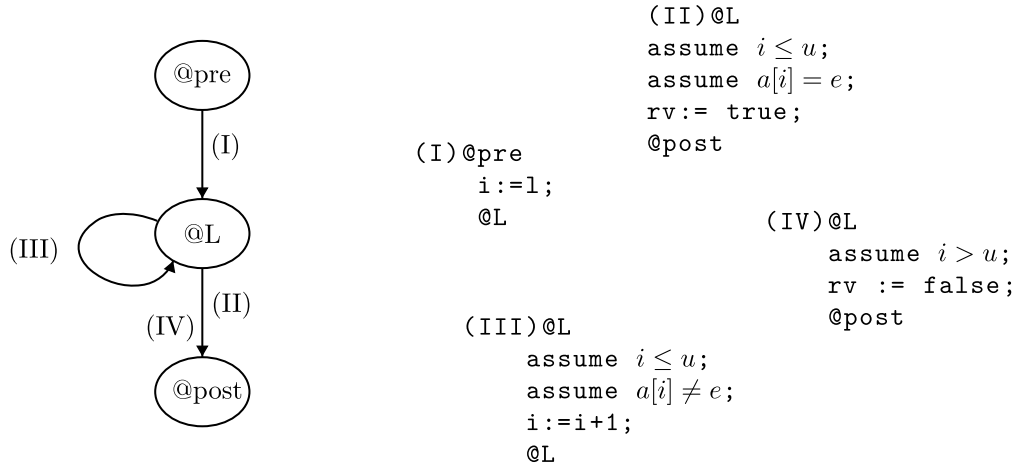


Figura 7: grafo dei cammini di base. Facendo riferimento all'invariante di ciclo @L, $\forall j: j < i$ perché dobbiamo ancora testare i . Bisogna verificare l'invariante interamente. Ad esempio nel caso di $l = i$ si ha che l'implicazione è vera perché la condizione sufficiente è falsa.

Per costruire i cammini di base, prima si traduce il for nel while come in fig. 8, si fa una sorta di “ricerca in profondità”, considerando le condizioni di ciclo e la condizione di guardia (fig. 7). **La decomposizione del programma in cammini di base è un'attività automatizzabile.**

Trasformatore di predicato Sia L un linguaggio di formule. Un trasformatore di predicato p è il prodotto cartesiano

$$p: L \times \langle \text{stmts} \rangle \longrightarrow L$$

dove $\langle \text{stmts} \rangle$ sono istruzioni. Quindi il trasformatore di predicato restituisce una formula in uscita che è il risultato della trasformazione della formula in ingresso rispetto ad un blocco di istruzioni $\langle \text{stmts} \rangle$.

Weakest pre-condition È un trasformatore di predicato, è una pre-condizione più debole che

$$wp(F, S)$$

<pre> for(<init>; <cond>; <inc>){ <body> } </pre>	$\longrightarrow$	<pre> <init> @I:F while(<cond>){ <body> <inc> } </pre>
---	-------------------	--

Figura 8: trasformazione del for nel while.

prende una formula e delle istruzioni e produce una formula. La $wp(F, S)$ è la pre-condizione più debole tale che F è vera dopo aver eseguito S .

Stato del programma È un assegnamento al PC¹² e a tutte le variabili e i parametri della funzione. Lo stato del programma si estende in un'interpretazione. Sia s uno stato del programma. Se siamo in uno stato $s \models wp(F, S)$ e $s \xrightarrow{S} s'$ e s' è un altro stato del programma allora $s' \models F$.

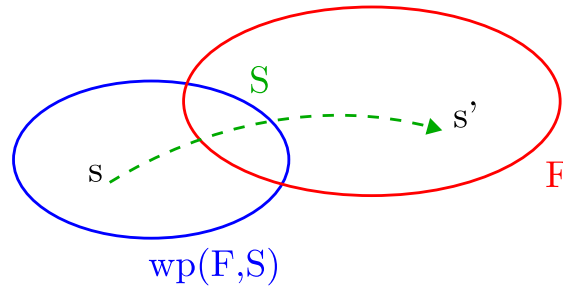


Figura 9: s è lo stato in cui è vera $wp(F, S)$ e s' è lo stato in cui è vera F . S è uno statement del programma ($S \in \langle \text{stmts} \rangle$) e F è una formula della logica del primo ordine.

1. $wp(F, \text{assume } C) = C \implies F \quad \frac{C \implies F}{F} \quad C$ che è il modus ponens.
2. $wp(F[x], x := e) = F[x \leftarrow e]$
 - $F[x]$: formula dove x compare
 - $F[x \leftarrow e]$: formula dove a x si sostituisce e .

Esempio

```

@F:  x ≥ 0
x := x + 1;
@G:  x + 1 ≥ 0

```

3. $wp(F, S_1; S_2) = wp(wp(F, S_2), S_1)$. Generalizzando $wp(F, S_1; \dots; S_n) = wp(wp(F, S_n), S_1; S_2; \dots; S_{n-1})$.
Se la formula $@F$ all'inizio del cammino è vera, allora è vera la pre-condizione più debole della formula alla fine del cammino rispetto alle istruzioni del cammino. Questa è una forma di

¹²Program Counter.

Tornando ad un cammino di base generico

```
@F
S1
S2
...
Sn
@G
```

qual'è la condizione di verifica per questo cammino? Si calcola la pre-condizione più debole $wp(G, S_1; \dots; S_n)$. La condizione di verifica sarà

$$wp(G, S_1; \dots; S_n) \models F \implies wp(G, S_1; \dots; S_n)$$

che equivale a risalire il cammino da @G a @F.

```
@pre: T
@post: sorted(rv , 0, |rv| - 1)
int[] BubbleSort(int[] a0) {
    int[] a := a0 ;
    for @L1:
        (int i := |a| - 1; i > 0; i := i - 1) {
            for @L2:
                (int j := 0; j < i; j := j + 1) {
                    if (a[j] > a[j + 1]) {
                        int t := a[j];
                        a[j] := a[j + 1];
                        a[j + 1] := t;
                    }
                }
            }
        }
    return a;
}
```

Riferimenti bibliografici

- [itf] it.frwiki.wiki. *Teorema di Rice*. URL: https://it.frwiki.wiki/wiki/Th%C3%A9or%C3%A8me_de_Rice.
- [Wik] Wikipedia. *Teorema di Rice*. URL: https://it.wikipedia.org/wiki/Teorema_di_Rice.